

Title:

Build an Advanced Training Loop: Mini-Batch Gradient Descent + Momentum + Learning-Rate Decay

Objective:

Enhance your mini-batch neural network implementation by adding **momentum** and **learning-rate decay**.

These improvements significantly stabilize and accelerate training, especially for deeper networks.

Requirements:

1. Neural Network Structure

- Same 2-layer neural network used in Programs 1 & 2
- Use sigmoid activation everywhere
- Solve XOR classification

2. New Features to Implement

a) Momentum

- Maintain a velocity vector for every weight
- Update rule:

```
v = (momentum * v) - (learning_rate * gradient)
w = w + v
```

- Typical momentum = 0.9

b) Learning-Rate Decay

Implement **time-based decay**:

```
lr(epoch) = lr_initial / (1 + decay * epoch)
```

- decay value example: 0.001
- Must update the learning rate every epoch

3. Training Method

- Use mini-batch gradient descent
- Accumulate gradients per batch
- Apply weight updates using **momentum + decayed learning rate**

4. Expected Output

- Print:
 - epoch number
 - current learning rate
 - training loss
- After training, show predictions for the XOR dataset

5. Goal

Understand how:

- Momentum smooths noisy gradients
- Learning-rate decay prevents overshooting
- Mini-batch GD stabilizes learning
- All three used together mimic training dynamics of modern deep networks.